Dart Programming Language

RegExp

```
void main() {
  String text = "My Roll Number is: 101";
  RegExp exp = RegExp(r'\d{3}');   ('\\d{3}')
  print(exp.hasMatch(text));
}
// true / false
```

\n → NextLine
\t → Tab Space
\\ → \d

https://

```
void main() {
  RegExp exp = RegExp(r'hello');
  print(exp.hasMatch('hello world'));
}
// true
```

Flags in RegExp :

1) caseSensitive :

```
Void main() {
  RegExp exp = RegExp(r'HELLO', caseSensitive: false);
  print(exp.hasMatch('Hello World'));
}
// false   // true
```

2) Unicode :

```
void main() {
  String text = "Welcome to কোডার ক্লাব বাংলা";
  RegExp exp = RegExp(r"\p{L}", unicode: true);
  print(exp.hasMatch(text));
}
// false
```

true

3) multiLine :

```
void main(){
  String text = '''
  apple
  Orange
  Grapes
  ''';
```

text = "apple\nOrange\nGrapes"

```
RegExp exp = RegExp(r'^[aeiouAEIOU].*[aeiouAEIOU]$', multiLine: true);
print(exp.hasMatch(text)); // false
}
```

4) dotAll :

```
void main(){
String text = "Start\nmiddle\nEnd";
RegExp exp = RegExp(r'Start.*End', dotAll: true);
print(exp.hasMatch(text)); // false
}
```

dotAll: true → true

Pattern	Meaning
1) .	Any Character Except newline
2) \d	0-9 / Digit
3) \D	Non-Digit
4) \w	Word character
5) \W	Non-Word
6) \s	White space
7) \S	Non-white space
8) *	0 or more

9) +

10) ?

11) {n}

12) {n,}

13) {n, m}

14) ^

15) $

16) [ ]

17) [^]

18) ( )

1 or more

optional 0 or 1

Exactly n

n or more

Range (n - m)

Start

End

character class

NOT

Group

— X —

1) hasMatch() ⇄ true ✓ / false ✓

2) firstMatch() → first Matched value

3) allMatches() → all Matches

4) group(0) → full matched text

5) group(1) → first Capture Group

6) forEach() → iterate

7) for-in → loop

8) .map() → transform results

* firstMatch():

```
void main () {
String text = "My numbers are 123 and 456";
RegExp exp = RegExp (r'\d+');
RegExpMatch? match = exp.firstMatch (text);
print (match); // Instance of '_RegExpMatch'
print (match?.group(0)); // 123
}
```

allMatches():-

```
void main () {
String text = 'My numbers are 123 and 456';
RegExp exp = RegExp (r'\d+');
Iterable<RegExpMatch> matches =
exp.allMatches (text);
print (matches); // (Instance of '_RegExpMatch',
                    Instance of '_RegExpMatch')
```

// for.in Loop

```
for(var match in matches){
print(match.group(0)); // 123
                          456
}
```

// forEach() :

```
matches.forEach((match) {
  print(match.group(0)); // 123
                         // 456
});
```

// .map() :

```
Iterable<String> modified = matches.
    map((match) {
      return match.group(0)!;
    });
print(modified); // (123, 456)
```

* group(0/1/2/3)

```
void main() {
String text = 'Flutter_Dart_Firebase';
RegExp exp = RegExp(r'(\w+)\s(\w+)\s(\w+)');
Iterable<RegExpMatch> matches = exp.allMatches
                                (text);
for(var match in matches) {
  print(match.group(0));
  print(match.group(1));
  print(match.group(2));
  print(match.group(3));
}
```

* Character Classes

```
void main () {
RegExp exp = RegExp(r'[abc]');
print (exp.hasMatch('apple')); //true
print (exp.hasMatch('dog')); // false
}
```

* Matches lowercase Letters

```
void main () {
RegExp exp = RegExp(r'[a-z]');
print (exp.hasMatch ('Banana')); //true
}
```

* Matches Uppercase Letters

```
void main() {
RegExp exp = RegExp(r'[A-Z]');
print (exp.hasMatch('Hello World'));
                                //true
}
```

* Matches digits

```
void main() {
RegExp exp = RegExp(r'[0-9]');
print (exp.hasMatch("12345abc")); // true
}
```

* Combined Example

```
void main () {
RegExp exp = RegExp(r'[A-Za-z0-9]');
```

```
print(exp.hasMatch('Rahul01')); //true
print(exp.hasMatch('* # %')); //false
}
```

* Negated Character Class [^ ___]

```
void main() {
RegExp exp = RegExp(r'[^0-9]');
print(exp.hasMatch('abc')); //true
}
```

* special Character Class (\d) → Digit [0-9]

```
void main () {
RegExp exp = RegExp(r'\d');
print(exp.hasMatch("Moon007");
//true
}
```

\D → Non-digit [^0-9]

```
void main () {
RegExp exp = RegExp(r'\D');
print(exp.hasMateh("Rahul"); //true
}
```

\w → word character [A-Za-z0-9_]

```
void main () {
RegExp exp = RegExp(r'\w');
```

print(exp.hasMatch("Ball")); // true ✓
print(exp.hasMatch("# @ %")); // false

}

\W → Non-word character [^A-Za-z0-9_]

void main() {
RegExp exp = RegExp(r"\W");
print(exp.hasMatch("# @ %")); // true
print(exp.hasMatch("Rahul")); // false

}

\s → whitespace (space / tab space / newline)

void main() {
RegExp exp = RegExp(r'\s');
print(exp.hasMatch("Flutter")); // false

}

\S → Non-whitespace

void main() {
RegExp exp = RegExp(r"\S");
print(exp.hasMatch("Flutter")); // true

}

* Dot . (Any Character except NewLine)

```
void main () {
RegExp exp = RegExp (r" a.c ");
print( exp. hasMatch(" abc "));  // true
print (exp. hasMatch( " axc "));  // true
}
```

* Quantifiers

  * * → 0 or more

```
void main () {
Print ( RegExp (r' ab* '). hasMatch (' a '));  //true
print (RegExp (r' ab* '). hasMatch (' ab '));  // true
Print (RegExp (r' ab* '). hasMatch (' abb '));  // true
}
```

  * + → 1 or more

```
void main () {
print( RegExp (r' ab+ '). hasMatch (" a "));  // false
print (RegExp (r' ab+ '). hasMatch (" ab "));  // true
Print (RegExp (r' ab+ '). hasMatch (" abb "));  // true
}
```

  * ? → optional (0 or 1)

```
void main () {
Print( RegExp (r' colou? r'). hasMatch('colour is Red'));
                                                  // true
Print (RegExp (r' colou? r'). hasMatch('color is Blue'));
                                                  // true
}
```

{n} → exactly n times

```
void main () {
print (RegExp (r'\d{2}/\d{2}/\d{4}').hasMatch
("Today's Date is : 01/06/2026"));
                                    // true
}
```

{n,} → At least n times

{4,} → 4, 5, 6, 7, 8, ...

```
void main () {
print (RegExp (r'\w{2,}').hasMatch ("d"));
                                    // false
print (RegExp (r'\w{2,}').hasMatch ("dogHoose"));
                                    // true
}
```

[A-Z/a-z/0-9/_]

{n, m} → between n and m

{2, 4} → (2, 3, 4)

```
void main(){
print( RegExp(r'\d{2,4}').hasMatch(
                "Number is : 3"));
print(RegExp(r'\d{2,4}').hasMatch// false
        ('Number is : 32243'));
                                  //true
```

Anchors (^ → start , $ → End)

```
print(RegExp(r"^\d{2,4}$").hasMatch
        ("32243"));
                                  // false
print(RegExp(r"^\w").hasMatch('Hello123"));
                                  //true
print(RegExp(r"^\w").hasMatch("_Hello"));
                                  //true
print(RegExp(r'^\w').hasMatch("*Hello"));
                                  // false
print(RegExp(r'\w$').hasMatch("*Hello"));
                                  //true
Print(RegExp(r'\w$').hasMatch("Hello%"));
                                  // false
```

print(RegExp(r'^.+$').hasMatch("hello")); //true

print(RegExp(r"^.+$").hasMatch(" ")); //false

* Alternation (|)

void main(){

Print(RegExp(r'cat|dog').hasMatch("I love cats")); //true

Print(RegExp(r'^(6|7|8|9)\d{9}$').hasMatch("7001126990")); //true

* Escaping Special Characters (. * + ? $ ^) need escaping using \

Print(RegExp(r'^\w\.\w$').hasMatch('google.com')); //true

Print(RegExp(r'^(\+91|91)(6|7|8|9)\d{9}$').hasMatch("+91 933 201 4590"));

print(RegExp(r'^\+?91(6|7|8|9)\d{9}$').hasMatch("91 9333 261660")); //true

word boundary and non word boundary

\b ✓ \B ✓

```
void main(){
print(RegExp(r'\bcat\b').hasMatch("I love cat")); // true
print(RegExp(r'\bcat\b').hasMatch('concatenation')); // false
}

print(RegExp(r'\Bcat\B').hasMatch("I love cat")); // false
print(RegExp(r'\Bcat\B').hasMatch("concatenation")); // true
```

- Capturing Group (...) → Match, remember
- Non Capturing Group (?:...) Match,

```
void main() {
  var match = RegExp(r'(\w+)-(\d+)').firstMatch('John-25');
  print(match?.group(0)); // John-25
  print(match?.group(1)); // John
  print(match?.group(2)); // 25
}

void main() {
  var match = RegExp(r'(?:cat|dog)')
      .firstMatch('dogs are animal');
  print(match?.group(0)); // dog
}
```

greedy and lazy

Greedy → " Match as much as possible.

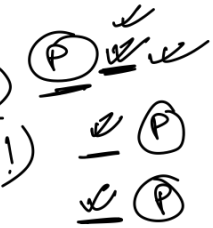
lazy → Match as little as possible

```
void main() {
String text= '<name> John </name>';
RegExp exp = RegExp(r'<.*>');
print (exp.firstMatch(text)?.group(0));
                    // <name> John </name>
}
```

```
void main () {
String text = "<name> John </name>";
RegExp exp = RegExp(r'<.*?>');
print (exp.firstMatch(text)?.group(0));
                    // <name>
}
```

* Lookaheads → Positive (?=)
  Negative (?!)

* Look behinds → Positive (?<=)
  Negative (?<!)




```
void main(){
String text = " 100tk  200tk  300 ";
RegExp exp = RegExp(r'\d+(?=tk)');
print( exp.allMatches(text).map((e) => e.group(0)).
                    toList());
}
```

[100, 200]

```
RegExp exp = RegExp(r'\b\d+\b(?!tk)');
print (exp.allMatches(text).map((e) => e.group(0)).toList());
```

```
}   //[300]

void main() {
String text = "\$100  \$200  300";
RegExp exp = RegExp(r'(?<=\$)\d+');
print(exp.allMatches(text).map((e) => e.group(0)).toList());
}                                   [100, 200]

RegExp exp = RegExp(r'(?<!\$)\b\d+\b');
print(exp.allMatches(text).map((e) => e.group(0))
                          .toList());
}                                   [300]
```

* replaceAll

```
void main() {
String text = 'I love cats. Cats are cute.';
String result = text.replaceAll(
RegExp(r'cats', caseSensitive: false), 'dogs');
print(result);
}          // I love dogs. dogs are cute.
```

* Split

```
void main() {
String text = "apple, banana; orange mango, , horse;
                                          moon";
List text1 = text.split(RegExp(r'[,;\s]+'));
print(text1);
} //[apple, banana, orange, mango, horse, moon]
```

* Match As prefix

```
void main() {
String text = "Hello World";
RegExp exp = RegExp(r'Hello');
Match? match = exp.matchAsPrefix(text);
print(match?.group(0));
}
```